

IF25-22017

PENGEMBANGAN APLIKASI MOBILE

PERTEMUAN 13

Project Sprint 3: Advanced Features

API Integration, Search, Offline Support, dan Extra Features

Program Studi Teknik Informatika
Institut Teknologi Sumatera
Tahun Akademik Genap 2025/2026



SPRINT PROGRESS

W11	Sprint 1: Planning & Setup	✓
W12	Sprint 2: Core Features	✓
W13	Sprint 3: Advanced Features	◀ NOW
W14	Sprint 4: Polish & Testing	
W15	Sprint 5: Final Preparation	
W16	UAS: Final Demo Day	

🎯 Sprint 3 Goal: Tambah advanced features - API, Search, Offline, dan fitur tambahan

OUTLINE PERTEMUAN

01

Sprint 2 Review

Demo dan feedback

02

API Integration

Ktor Client dan REST API

03

Search & Filter

Implementasi pencarian

04

Offline Support

Caching dan sync strategies

05

Extra Features

Dark mode, animations, dll

06

Sprint 3 Tasks

Work session dan deliverables

BAGIAN 1

SPRINT 2 REVIEW

Demo dan Feedback

SPRINT 2 CHECKLIST

☐ Minimal 3 working screens (Home, Detail, Add/Edit)

☐ Navigation dengan arguments working

☐ Repository pattern implemented

☐ Local storage (SQLDelight/DataStore) working

☐ CRUD operations functional

☐ UI States (Loading, Success, Error, Empty)

 **Jika Sprint 2 belum selesai, prioritaskan menyelesaikannya sebelum advanced features!**

BAGIAN 2

API INTEGRATION

Ktor Client dan REST API

KAPAN MENGGUNAKAN API?

✓ Gunakan API jika:

- Data perlu diakses dari multiple devices
- Data sering berubah (news, weather)
- Butuh data eksternal (public APIs)
- Multi-user collaboration
- Backend sudah tersedia

⚡ Local-only jika:

- Personal app (notes, todos)
- Tidak perlu sync antar device
- Offline-first adalah prioritas
- Tidak ada backend
- Simplicity > features

Public APIs untuk latihan:

JSONPlaceholder	jsonplaceholder.typicode.com	Fake REST API
OpenWeatherMap	openweathermap.org/api	Weather data
NewsAPI	newsapi.org	News articles
The Movie DB	themoviedb.org/documentation/api	Movies data

KTOR CLIENT SETUP

di/NetworkModule.kt

```
val networkModule = module {  
    single {  
        HttpClient {  
            install(ContentNegotiation) {  
                json(Json {  
                    ignoreUnknownKeys = true  
                    prettyPrint = true  
                    isLenient = true  
                })  
            }  
            install(Logging) {  
                logger = Logger.DEFAULT  
                level = LogLevel.INFO  
            }  
            install(HttpTimeout) {  
                requestTimeoutMillis = 15000  
                connectTimeoutMillis = 15000  
            }  
            defaultRequest {  
                url("https://api.example.com/")  
                contentType(ContentType.Application.Json)  
            }  
        }  
    }  
    single<ApiService> { ApiServiceImpl(get()) }  
}
```

API SERVICE IMPLEMENTATION

```
interface ApiService {
    suspend fun getItems(): List<ItemDto>
    suspend fun getItem(id: Long): ItemDto
    suspend fun createItem(item: ItemDto): ItemDto
    suspend fun updateItem(id: Long, item: ItemDto): ItemDto
    suspend fun deleteItem(id: Long)
}

class ApiServiceImpl(private val client: HttpClient) : ApiService {

    override suspend fun getItems(): List<ItemDto> {
        return client.get("items").body()
    }

    override suspend fun getItem(id: Long): ItemDto {
        return client.get("items/$id").body()
    }

    override suspend fun createItem(item: ItemDto): ItemDto {
        return client.post("items") {
            setBody(item)
        }.body()
    }

    override suspend fun deleteItem(id: Long) {
        client.delete("items/$id")
    }
}
```

REPOSITORY DENGAN API + LOCAL

```
class ItemRepositoryImpl(
    private val apiService: ApiService,
    private val database: AppDatabase
) : ItemRepository {

    override fun getAllItems(): Flow<List<Item>> {
        return database.itemQueries.selectAll()
            .asFlow()
            .mapToList(Dispatchers.IO)
            .map { it.map { entity -> entity.toDomain() } }
    }

    override suspend fun refreshItems() {
        withContext(Dispatchers.IO) {
            try {
                val remoteItems = apiService.getItems()
                // Clear and insert fresh data
                database.itemQueries.deleteAll()
                remoteItems.forEach { dto ->
                    database.itemQueries.insert(
                        dto.id, dto.title, dto.description, dto.createdAt
                    )
                }
            } catch (e: Exception) {
                // Handle network error - local data still available
                throw e
            }
        }
    }
}
```

API ERROR HANDLING

Error Handling

```
sealed class NetworkResult<out T> {
    data class Success<T>(val data: T) : NetworkResult<T>()
    data class Error(val message: String, val code: Int? = null) : NetworkResult<Nothing>()
    data object Loading : NetworkResult<Nothing>()
}

suspend fun <T> safeApiCall(apiCall: suspend () -> T): NetworkResult<T> {
    return try {
        NetworkResult.Success(apiCall())
    } catch (e: ClientRequestException) {
        NetworkResult.Error("Client error: ${e.response.status}", e.response.status.value)
    } catch (e: ServerResponseException) {
        NetworkResult.Error("Server error: ${e.response.status}", e.response.status.value)
    } catch (e: Exception) {
        NetworkResult.Error(e.message ?: "Unknown error")
    }
}
```



Common HTTP Errors:

- 400 Bad Request - Invalid data sent
- 401 Unauthorized - Auth required
- 404 Not Found - Resource doesn't exist
- 500 Server Error - Backend issue

BAGIAN 3

SEARCH & FILTER

Implementasi Pencarian

SEARCH DI VIEWMODEL

```
class HomeViewModel(private val repository: ItemRepository) : ViewModel() {

    private val _searchQuery = MutableStateFlow("")
    val searchQuery: StateFlow<String> = _searchQuery.asStateFlow()

    val uiState: StateFlow<HomeUiState> = combine(
        repository.getAllItems(),
        _searchQuery
    ) { items, query ->
        val filteredItems = if (query.isBlank()) {
            items
        } else {
            items.filter { item ->
                item.title.contains(query, ignoreCase = true) ||
                item.description.contains(query, ignoreCase = true)
            }
        }

        when {
            filteredItems.isEmpty() && query.isNotBlank() -> HomeUiState.NoResults(query)
            filteredItems.isEmpty() -> HomeUiState.Empty
            else -> HomeUiState.Success(filteredItems, query)
        }
    }.stateIn(
        scope = viewModelScope,
        started = SharingStarted.WhileSubscribed(5000),
        initialValue = HomeUiState.Loading
    )

    fun onSearchQueryChange(query: String) {
        _searchQuery.value = query
    }
}
```

SEARCH BAR UI

```
@Composable
fun SearchBar(
    query: String,
    onQueryChange: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    OutlinedTextField(
        value = query,
        onValueChange = onQueryChange,
        modifier = modifier.fillMaxWidth(),
        placeholder = { Text("Search...") },
        leadingIcon = { Icon(Icons.Default.Search, contentDescription = null) },
        trailingIcon = {
            if (query.isNotEmpty()) {
                IconButton(onClick = { onQueryChange("") }) {
                    Icon(Icons.Default.Clear, contentDescription = "Clear")
                }
            }
        },
        singleline = true,
        shape = RoundedCornerShape(12.dp)
    )
}
```

Debounce untuk API search:

Gunakan `.debounce(300)` sebelum API call untuk menghindari terlalu banyak request saat user mengetik.

FILTER OPTIONS

```
enum class SortOption {
    DATE_DESC, DATE_ASC, TITLE_ASC, TITLE_DESC
}

enum class FilterOption {
    ALL, COMPLETED, PENDING
}

data class FilterState(
    val sortBy: SortOption = SortOption.DATE_DESC,
    val filterBy: FilterOption = FilterOption.ALL
)

// Di ViewModel
private val _filterState = MutableStateFlow(FilterState())

val filteredItems = combine(
    repository.getAllItems(),
    _searchQuery,
    _filterState
) { items, query, filter ->
    items
        .filter { matchesSearch(it, query) }
        .filter { matchesFilter(it, filter.filterBy) }
        .sortedWith(getSortComparator(filter.sortBy))
}

fun updateSort(option: SortOption) {
    _filterState.update { it.copy(sortBy = option) }
}
```

BAGIAN 4

OFFLINE SUPPORT

Caching dan Sync Strategies

OFFLINE-FIRST STRATEGIES

Cache-First

Show cached data immediately, then refresh from API

Use case: News, social feeds

Network-First

Try API first, fallback to cache if offline

Use case: Real-time data

Cache-Only

Never use network, fully offline app

Use case: Personal notes, todos

Stale-While-Revalidate

Show cache, update in background, refresh UI

Use case: Best UX for most apps

CACHE-FIRST IMPLEMENTATION

```
class ItemRepositoryImpl(
    private val api: ApiService,
    private val db: AppDatabase
) : ItemRepository {

    // Always return local data as Flow
    override fun getAllItems(): Flow<List<Item>> {
        return db.itemQueries.selectAll()
            .asFlow()
            .mapToList(Dispatchers.IO)
            .map { it.map { e -> e.toDomain() } }
    }

    // Refresh from network, update local cache
    override suspend fun refresh(): RefreshResult {
        return withContext(Dispatchers.IO) {
            try {
                val remote = api.getItems()
                db.transaction {
                    db.itemQueries.deleteAll()
                    remote.forEach { dto ->
                        db.itemQueries.insert(dto.toEntity())
                    }
                }
                RefreshResult.Success
            } catch (e: Exception) {
                RefreshResult.Error(e.message ?: "Network error")
            }
        }
    }
}

sealed class RefreshResult {
    data object Success : RefreshResult()
    data class Error(val message: String) : RefreshResult()
}
```

PULL TO REFRESH

```
@Composable
fun HomeScreen(viewModel: HomeViewModel = koinViewModel()) {
    val uiState by viewModel.uiState.collectAsState()
    val isRefreshing by viewModel.isRefreshing.collectAsState()

    PullToRefreshBox(
        isRefreshing = isRefreshing,
        onRefresh = { viewModel.refresh() }
    ) {
        when (val state = uiState) {
            is HomeUiState.Loading -> LoadingIndicator()
            is HomeUiState.Success -> {
                ItemList(items = state.items)
            }
            is HomeUiState.Error -> {
                ErrorWithRetry(message = state.message, onRetry = { viewModel.refresh() })
            }
        }
    }
}
```



ViewModel refresh function:

```
fun refresh() {
    viewModelScope.launch {
        _isRefreshing.value = true
        repository.refresh()
        _isRefreshing.value = false
    }
}
```

BAGIAN 5

EXTRA FEATURES

Dark Mode, Animations, dan Lainnya

DARK MODE SUPPORT

```
// Theme.kt
enum class ThemeMode { LIGHT, DARK, SYSTEM }

@Composable
fun AppTheme(
    themeMode: ThemeMode = ThemeMode.SYSTEM,
    content: @Composable () -> Unit
) {
    val darkTheme = when (themeMode) {
        ThemeMode.LIGHT -> false
        ThemeMode.DARK -> true
        ThemeMode.SYSTEM -> isSystemInDarkTheme()
    }

    MaterialTheme(
        colorScheme = if (darkTheme) darkColorScheme() else lightColorScheme(),
        typography = AppTypography,
        content = content
    )
}

// Save preference with DataStore
class SettingsRepository(private val datastore: DataStore<Preferences>) {
    val themeMode: Flow<ThemeMode> = datastore.data.map { prefs ->
        ThemeMode.valueOf(prefs[THEME_KEY] ?: ThemeMode.SYSTEM.name)
    }

    suspend fun setThemeMode(mode: ThemeMode) {
        datastore.edit { it[THEME_KEY] = mode.name }
    }
}
```

SIMPLE ANIMATIONS

Fade Animation

```
@Composable
fun FadeInContent(visible: Boolean) {
    AnimatedVisibility(
        visible = visible,
        enter = fadeIn() + expandVertically(),
        exit = fadeOut() + shrinkVertically()
    ) {
        // Content here
    }
}
```

Scale Animation

```
@Composable
fun ScaleOnPress() {
    var pressed by remember { mutableStateOf(false) }
    val scale by animateFloatAsState(
        targetValue = if (pressed) 0.95f else 1f,
        animationSpec = spring()
    )

    Box(
        modifier = Modifier
            .scale(scale)
            .pointerInput(Unit) {
                detectTapGestures(
                    onPress = { pressed = true; tryAwaitRelease(); pressed = false }
                )
            }
    )
}
```

List Animation

```
LazyColumn {
    items(items, key = { it.id }) { item ->
        ItemCard(
            item = item,
            modifier = Modifier.animateItem() // Built-in list animation
        )
    }
}
```

MORE FEATURE IDEAS

Notifications	Local notifications untuk reminders	+5%
Share Feature	Share content ke apps lain	+3%
Biometric Auth	Fingerprint/Face unlock	+5%
Export Data	Export ke PDF/CSV	+5%
Widgets	Home screen widgets (Android)	+8%
Accessibility	Content descriptions, large text	+3%

BAGIAN 6

SPRINT 3 TASKS

Work Session dan Deliverables

SPRINT 3 GOALS

P0	Search/Filter Search functionality dalam app
P0	API Integration OR Enhanced Local REST API jika applicable, atau enhanced local features
P1	Offline Support App tetap usable tanpa internet
P1	Additional Screen Settings, profile, atau feature screen
P2	UI Polish Consistent styling, better UX
P2	1+ Bonus Feature Dark mode, animations, dll

DELIVERABLES SPRINT 3

Bobot: 5% | Deadline: Sebelum Pertemuan 14

Deliverables:

- ☐ Search/filter functionality working
- ☐ API integration ATAU enhanced local features
- ☐ At least 1 additional screen (Settings/Profile)
- ☐ Offline support (app usable tanpa internet)
- ☐ At least 1 bonus feature implemented
- ☐ All core features dari Sprint 2 tetap working

Submission:

- GitHub repo updated dengan advanced features
- Video demo (1-2 menit): show search, offline mode, bonus features
- CI passing, code quality maintained

RUBRIK PENILAIAN SPRINT 3

Komponen	Bobot	Kriteria
Search/Filter	25%	Working search, responsive, good UX
API/Enhanced Local	25%	Proper integration, error handling
Offline Support	20%	App usable offline, graceful degradation
Additional Screen	15%	Settings/Profile functional
Bonus Features	15%	At least 1 bonus implemented well
★ Bonus: Extra features (+10%)		⚠ Late: -10%/day



WORK SESSION

Waktu: Sisa pertemuan (~60 menit)

1	10 min	Review Sprint 2, identify what's missing
2	10 min	Plan Sprint 3 - which features to implement
3	30 min	Start implementing - search, API, atau bonus features
4	10 min	Commit & push, update README

Target hari ini:

- Search functionality skeleton ready
- API service atau enhanced features planned
- At least 1 bonus feature started

PREVIEW: SPRINT 4 (MINGGU DEPAN)

Sprint 4: Polish & Testing

- **Bug Fixes**

Fix all known bugs dan edge cases

- **UI Polish**

Consistent styling, spacing, typography

- **Testing**

Unit tests, UI tests, coverage goal

- **Performance**

Optimize slow screens, reduce lag

🔥 **Persiapan: Complete Sprint 3, list all known bugs, review Week 10 (Testing)**

TIPS SPRINT 3



Don't over-scope

Pilih 2-3 advanced features, selesaikan dengan baik



API fallback

Always have offline fallback jika API fails



Test edge cases

Empty search, network error, slow connection



Test on real device

Emulator tidak selalu representatif



Polish matters

Small details make big difference in UX



Document your APIs

Note down endpoints, keys di README

RESOURCES

Docs

Ktor Client

ktor.io/docs/client.html

Docs

Compose Animation

developer.android.com/jetpack/compose/animation

Guide

Offline-First Apps

developer.android.com/topic/architecture/data-layer/offline-first

API

Public APIs List

github.com/public-apis/public-apis



SESI TANYA JAWAB

Ada pertanyaan tentang Sprint 3?

CHECKLIST SEBELUM PULANG

- ☐ Sprint 2 complete atau on-track
- ☐ Sprint 3 features planned
- ☐ Search functionality started
- ☐ API/enhanced features strategy decided
- ☐ Bonus feature(s) chosen
- ☐ Code pushed ke repository
- ☐ Paham deadline dan deliverables Sprint 3



Level up your app with advanced features!

Make It Better! ✨

Sprint 3: Advanced Features

*"Details matter, it's worth waiting
to get it right."*

- Steve Jobs

See you next week for Sprint 4! 🎯

Terima Kasih

Happy Coding! 🚀

3 Weeks to Demo Day!

Sprint 3: Advanced Features

Sprint 4: Polish & Testing

Sprint 5: Final Preparation

UAS: Demo Day 🎉

Keep pushing! 💪